

Bangladesh University of Engineering & Technology

Course No.: CSE 318

Course Name: Artificial Intelligence Sessional

**Topic: Performance Evaluation of the Custom
Heuristic Used in “Weighted A^* Search” for
15-Puzzle**

Name: Md. Shoyaib Rezwan

Student ID: 2205014

July 6, 2026

1 Introduction

This report details the offline assignment for solving the N -puzzle problem using the A^* search algorithm. The objective is to efficiently arrange a $k \times k$ grid (4×4 for the 15-puzzle) into numerical order using a blank tile. The assignment requires the implementation of four standard heuristics (Hamming, Manhattan, Euclidean, and Linear Conflict) as well as a distinct Custom Heuristic.

This report specifically evaluates the performance of the Custom Heuristic across a batch of 10 different board configurations. To verify the accuracy of the generated results and minimum move counts, the configurations were cross-referenced using the on-line solver at <https://www.dcode.fr/15-puzzle-solver>. Furthermore, a strict computational threshold of 4,000,000 nodes was implemented; if the search algorithm exceeds this limit before reaching the goal, a “Resource limit exceeded” output is triggered.

2 Custom Heuristic: Linear Conflict + $2 \times$ Corner Conflict

The custom heuristic implemented for this evaluation is defined as:

$$h(n) = \text{Linear Conflict} + 2 \times \text{Corner Conflict}$$

2.1 Why this heuristic?

The Linear Conflict heuristic alone is a powerful extension of Manhattan Distance, accounting for tiles that are in their correct row or column but in the wrong order. However, it can be further improved by addressing “Corner Conflicts.” A corner conflict occurs when a tile is correctly placed in a corner, but its immediate adjacent tiles are conflicting and must be temporarily displaced to solve the board.

This combined heuristic was selected because designing a completely novel, mathematically admissible, and highly efficient heuristic from scratch is exceedingly difficult. By building upon an already existing and proven heuristic framework, we ensure that the search remains highly informed and drastically reduces the number of explored nodes without violating admissibility at $W = 1.0$.

3 Metrics Evaluation Table

The following tables map the performance of the Weighted A^* algorithm using priority function $f(n) = g(n) + W \cdot h(n)$. The Overestimation percentage is calculated as:

$$\text{Overestimation} = \left(\frac{\text{Solution Cost} - \text{Minimum Moves}}{\text{Minimum Moves}} \right) \times 100\%$$

$$\text{Performance Metric} = \left(\frac{\text{Solution Cost}}{\text{Explored Nodes}} \right) \times 100\%$$

3.1 Test Case 1

Input Grid:

```
1  2  3  4
5  6  7  8
9 10  0 12
13 14 11 15
```

Minimum moves to solve: 2

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	3	2	66.6667%	0.00%
1.2	3	2	66.6667%	0.00%
2.0	3	2	66.6667%	0.00%
5.0	3	2	66.6667%	0.00%

3.2 Test Case 2

Input Grid:

```
1  2  3  4
5  6  7  8
9 10 11 12
0 13 14 15
```

Minimum moves to solve: 3

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	4	3	75.0000%	0.00%
1.2	4	3	75.0000%	0.00%
2.0	4	3	75.0000%	0.00%
5.0	4	3	75.0000%	0.00%

3.3 Test Case 3

Input Grid:

```
1  2  3  4
0  5  6  7
9 10 11  8
13 14 15 12
```

Minimum moves to solve: 5

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	6	5	83.3333%	0.00%
1.2	6	5	83.3333%	0.00%
2.0	6	5	83.3333%	0.00%
5.0	6	5	83.3333%	0.00%

3.4 Test Case 4

Input Grid:

5 1 2 4
9 6 7 3
13 10 11 8
0 15 14 12

Minimum moves to solve: 23

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	1297	23	1.7733%	0.00%
1.2	695	23	3.3093%	0.00%
2.0	451	23	5.0997%	0.00%
5.0	528	39	7.3863%	69.56%

3.5 Test Case 5

Input Grid:

1 10 7 11
9 6 14 2
13 12 4 3
5 15 8 0

Minimum moves to solve: 40

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	7848	40	0.5096%	0.00%
1.2	3735	40	1.0709%	0.00%
2.0	8150	46	0.5644%	15.00%
5.0	450	52	11.5556%	30.00%

3.6 Test Case 6

Input Grid:

12 5 3 1
8 0 14 6
11 2 7 15
4 9 13 10

Minimum moves to solve: 54

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	847076	54	0.0063%	0.00%
1.2	34173	54	0.1580%	0.00%
2.0	1509	60	3.9761%	11.11%
5.0	361	84	23.2687%	55.55%

3.7 Test Case 7

Input Grid:

7 14 2 11
4 13 1 9
0 8 10 5
15 3 6 12

Minimum moves to solve: 52

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	62955	52	0.0825%	0.00%
1.2	4093	52	1.2704%	0.00%
2.0	1923	72	3.7441%	38.46%
5.0	444	100	22.5225%	92.31%

3.8 Test Case 8

Input Grid:

0 12 6 15
3 1 10 8
2 13 5 11
14 7 4 9

Minimum moves to solve: 46

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	44194	46	0.1040%	0.00%
1.2	20035	48	0.2395%	4.34%
2.0	1541	56	3.6340%	21.73%
5.0	411	80	19.4647%	73.91%

3.9 Test Case 9

Input Grid:

13 2 9 0
12 6 4 15
7 1 8 11
10 5 14 3

Minimum moves to solve: 51

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	709493	51	0.0071%	0.00%
1.2	72772	53	0.0728%	3.92%
2.0	5140	59	1.1478%	15.68%
5.0	3952	91	2.3026%	78.43%

3.10 Test Case 10

Input Grid:

```
0 12 9 13
15 11 10 14
3 7 2 5
4 8 6 1
```

Minimum moves to solve: 80 (Resource online, limits exceeded for optimal search)

Weight	Explored Nodes	Solution Cost	Performance Metric	Overestimation
1.0	-	-	-	-
1.2	-	-	-	-
2.0	3108	94	3.0244%	17.50%
5.0	505	136	26.9307%	70.00%

4 Discussion

4.1 Why does increasing weights reduce precision?

In standard A^* search ($W = 1.0$), the algorithm guarantees the mathematically optimal shortest path because the heuristic is strictly admissible (it never overestimates the cost to the goal). By applying a weight $W > 1.0$ to the heuristic function, the search prioritizes nodes that appear closer to the goal state over nodes that have a shorter path from the start. This creates a “Greedy” behavior. While it heavily prunes the search tree and radically reduces the number of explored nodes, it sacrifices admissibility, leading the algorithm to settle for sub-optimal, longer solution paths (lower precision).

4.2 Why do some solutions exceed the resource limit?

The 15-puzzle has an enormous state space of $\frac{16!}{2} \approx 10.4 \times 10^{12}$ reachable configurations. For highly scrambled boards (such as Input 10, which requires 80 optimal moves), standard unweighted A^* ($W = 1.0$) explores nodes exponentially. Even with an informed custom heuristic, the frontier size outpaces available memory, quickly hitting the predefined 4M node threshold. Applying weights ($W \geq 2.0$) forces the algorithm to bypass the exhaustive breadth-first nature of A^* , keeping the node count manageable enough to return a solution before the resource limit triggers.